

EduConnect Development Roadmap

Project Title

EduConnect: A Peer-to-Peer and Collaborative Learning System for Students

1. Project Overview

EduConnect is a web-based collaborative learning platform designed to help students:

- ask academic questions
- receive peer support
- join study groups
- access learning resources
- communicate in real time

The project will use:

- React.js (Frontend)
 - Django + Django REST Framework (Backend)
 - PostgreSQL (Database)
 - WebSockets / Django Channels (Real-time communication)
 - JWT Authentication
 - Agile Scrum Methodology
-

2. Development Methodology

The project will follow:

- Agile Scrum
- Object-Oriented Analysis and Design (OOAD)

Development will be divided into iterative sprints with continuous testing and feedback.

3. Minimum Viable Product (MVP)

The first version of EduConnect should include:

Core Features

- User Registration and Login
- JWT Authentication
- Question Posting
- Answer Submission
- Study Groups
- Real-time Chat
- Resource Repository
- Notifications

Features Deferred to Later Stages

- AI Features
 - Advanced Gamification
 - Mobile Application
 - Recommendation Engine
 - Analytics Dashboard
-

4. Team Structure

Backend Developer Responsibilities

- Django backend
- API development
- PostgreSQL database
- JWT authentication
- WebSockets
- Deployment

Frontend Developer Responsibilities

- React frontend
 - UI/UX
 - API integration
 - State management
 - Responsive design
-

5. Project Phases

PHASE 1 — Planning and Analysis

Objectives

- Define system requirements
- Identify core features
- Plan architecture

Tasks

- Define functional requirements
- Define non-functional requirements
- Create user stories
- Define project scope
- Setup GitHub repository

Deliverables

- Requirement specification document
 - Initial project plan
 - GitHub repository
-

PHASE 2 — System Design

Objectives

Design the structure of EduConnect before development begins.

Tasks

2.1 Database Design

Create the Entity Relationship Diagram (ERD).

Main Entities

- User
- Question
- Answer
- StudyGroup
- GroupMember
- Message
- Notification

- Resource
- Streak

2.2 UML Diagrams

Create:

- Use Case Diagram
- Sequence Diagram
- Activity Diagram
- Class Diagram
- ER Diagram
- System Architecture Diagram

2.3 Wireframes

Design:

- Login Page
- Registration Page
- Dashboard
- Question Feed
- Question Details
- Group Chat
- Profile Page

Tools

- Draw.io
- Lucidchart
- Figma

Deliverables

- ERD
- UML diagrams
- Wireframes
- System architecture document

PHASE 3 — Development Environment Setup

Objectives

Prepare all development tools and environments.

Tasks

Backend Setup

Install:

- Python
- Django
- PostgreSQL

Frontend Setup

Install:

- Node.js
- React.js

IDE

Use:

- Visual Studio Code
-

Backend Project Setup

Create Django Project

```
django-admin startproject config .
```

Create Apps

```
python manage.py startapp users
python manage.py startapp questions
python manage.py startapp groups
python manage.py startapp chat
python manage.py startapp notifications
```

Install Backend Dependencies

```
pip install django
pip install djangorestframework
```

```
pip install psycpg2
pip install django-rest-framework-simplejwt
pip install channels
pip install channels-redis
pip install django-cors-headers
```

Frontend Setup

Create React Project

```
npm create vite@latest frontend
```

Install Frontend Dependencies

```
npm install react-router-dom axios
```

Later Install

```
npm install socket.io-client
```

PHASE 4 — Database and Authentication

Objectives

Create the system foundation.

Tasks

PostgreSQL Configuration

- Create database
- Create database user
- Configure Django settings

Create Custom User Model

User Fields

- username
 - email
 - password
 - role
 - bio
 - streak
 - profile_picture
-

JWT Authentication

APIs

- Register API
- Login API
- Refresh Token API
- Profile API

Testing Tools

- Postman
 - Thunder Client
-

Frontend Authentication

Features

- Login page
- Registration page
- Protected routes
- JWT token handling

Deliverables

- Secure authentication system
-

PHASE 5 — Question and Answer System

Objectives

Develop the core collaborative learning module.

Models

Question

- title
- body
- subject
- author
- created_at

Answer

- question
 - body
 - author
 - is_best
 - created_at
-

APIs

- GET /questions
 - POST /questions
 - GET /questions/:id
 - POST /answers
-

Frontend Features

- Question feed
- Ask question form
- Question details page
- Search functionality
- Filters
- Mark best answer

Deliverables

- Functional Q&A system

PHASE 6 — Resource Repository

Objectives

Allow students to share academic resources.

Features

- Upload resources
- Share links
- Subject categorization
- Resource ratings
- Search resources

Deliverables

- Resource repository module
-

PHASE 7 — Study Groups

Objectives

Enable collaborative learning groups.

Models

StudyGroup

- name
- subject
- meeting_link
- created_by

GroupMember

- user
 - group
-

Features

- Create group
- Join group
- Leave group
- Group dashboard

Initial Approach

Meeting links may initially be manually added.

Deliverables

- Study group system
-

PHASE 8 — Real-Time Communication

Objectives

Implement live communication.

Technology

- Django Channels
- WebSockets

Features

- Real-time chat
- Live notifications
- Online users

Development Steps

1. Single chat room
2. Multi-room support
3. Notifications

Deliverables

- Functional real-time communication system
-

PHASE 9 — Notifications

Objectives

Provide instant updates to users.

Notifications

- New answer
- New message
- Group invitation
- Meeting reminder

Deliverables

- Notification system
-

PHASE 10 — Gamification

Objectives

Increase user engagement.

Features

- Daily streaks
- Points system
- Leaderboards
- Achievement badges

Example Rewards

- Answer question → +5 points
- Best answer → +10 points
- Daily login → +2 points

Deliverables

- Gamification module
-

PHASE 11 — Testing

Objectives

Ensure system quality and reliability.

Types of Testing

Unit Testing

Test individual components.

Integration Testing

Test communication between modules.

System Testing

Test full workflows.

User Acceptance Testing (UAT)

Allow real users to test the platform.

Example Test Cases

Test Case	Expected Result
User Login	JWT token generated
Post Question	Question stored
Send Message	Message received instantly
Join Group	User added successfully

Deliverables

- Test reports
 - Bug reports
 - UAT reports
-

PHASE 12 — UI/UX Improvement

Objectives

Improve usability and appearance.

Tasks

- Responsive design
- Better layouts
- Loading states
- Error handling
- Accessibility improvements

Deliverables

- Improved user experience
-

PHASE 13 — Security Implementation

Objectives

Protect the platform and user data.

Security Features

- Password hashing
- JWT validation
- Protected routes
- Role-based permissions
- CORS configuration
- API validation

Deliverables

- Secure application
-

PHASE 14 — Deployment

Objectives

Deploy the system online.

Backend Hosting Options

- Render
- Railway

Frontend Hosting Options

- Vercel
- Netlify

Database Hosting Options

- Neon PostgreSQL
 - Supabase PostgreSQL
-

Deployment Tasks

- Configure production settings
- Configure environment variables
- Deploy frontend
- Deploy backend
- Connect production database

Deliverables

- Live EduConnect platform
-

PHASE 15 — Final Documentation

Objectives

Prepare academic and technical documentation.

Required Documentation

Technical Documentation

- Architecture
- APIs
- Setup instructions

User Manual

- Registration guide
- Posting questions
- Joining groups

Testing Documentation

- Test cases
- Results
- Bug reports

Installation Guide

- Local setup
- Deployment setup

Deliverables

- Complete project documentation
-

PHASE 16 — Final Presentation Preparation

Objectives

Prepare for project defense and demonstration.

Prepare

- System demonstration
 - Architecture explanation
 - Methodology explanation
 - Challenges faced
 - Future improvements
-

6. Recommended Development Order

1. Planning
2. Database Design
3. Authentication
4. CRUD APIs
5. Frontend Integration
6. Study Groups
7. WebSockets
8. Notifications
9. Gamification
10. Testing
11. Deployment

7. Recommended GitHub Structure

```
educonnect/  
|  
├─ frontend/  
├─ backend/  
├─ docs/  
└─ README.md
```

8. Suggested Sprint Breakdown

Sprint	Focus
Sprint 1	Setup + Authentication
Sprint 2	Questions + Answers
Sprint 3	Study Groups
Sprint 4	Real-time Chat
Sprint 5	Notifications
Sprint 6	Gamification
Sprint 7	Testing + UI Improvements
Sprint 8	Deployment + Documentation

9. Recommended Learning Priorities

Before advanced features, ensure strong understanding of:

- Django REST Framework
 - React fundamentals
 - PostgreSQL
 - JWT Authentication
 - WebSockets
 - Git and GitHub
 - API integration
-

10. Final Notes

The most important principle throughout development is:

Build simple features first, then improve incrementally.

Focus first on:

- working backend
- correct database design
- authentication
- CRUD functionality
- stable APIs

Avoid spending excessive time on:

- advanced animations
- unnecessary complexity
- premature optimization

The primary goal is to deliver a functional, reliable and maintainable collaborative learning platform.